

Playwright + TypeScript + BDD Framework

+ AI & MCP Integration

A Modern, Scalable & Intelligent Automation Ecosystem

3-Month Batch Syllabus — 8 Modules — 2 Batch Options

Course Overview

Course	Playwright + TypeScript + BDD + AI & MCP Integration
Instructor	Hemant Gandhi Lead Automation Engineer (11+ Years)
Platform	Journey to Automation QodeBench
Level	Beginner to Advanced — QA Engineers & SDETs
Batch Options	Option A: Weekend — Sat & Sun, 8:00 PM – 10:00 PM IST Option B: Weekday — Mon/Tue/Wed, 7:15 AM – 9:00 AM IST
Duration	8 Modules 3 Months Both Options Available
Tech Stack	Playwright, TypeScript, Cucumber, Jenkins, GitHub Actions, Allure, Node.js, MySQL, Postman
AI Stack	GPT-4o / Claude / Gemini, @playwright/mcp Agent, Self-Healing Tests, AI Failure Analysis

Batch Options at a Glance

	Option A — Weekend Batch	Option B — Weekday Batch
Days	Saturday + Sunday	Monday + Tuesday + Wednesday
Per Session	8:00 PM – 10:00 PM IST	7:15 AM – 9:00 AM IST
Total Sessions	24 Sessions	39 Sessions
Total Weeks	12 Weeks	13 Weeks
Duration	3 Months	~3 Months
Best For	Working professionals — relax on weekdays	Early risers — get it done before work

Session Schedules

Detailed Module Syllabus

Module 01 — TypeScript Fundamentals for Test Automation

Write clean, typed, scalable test code from day one

TS Basics	– Types, interfaces, enums & generics – Functions, arrow functions, async/await & Promises – Modules — import/export patterns – tsconfig.json setup for Playwright projects
TS for Testers	– Typed page objects & step definitions – Type-safe API clients & response models – Null safety & optional chaining in test code
Dev Environment	– VS Code setup — ESLint, Prettier, Path aliases – Node.js & npm project structure – Running your first TypeScript test

Module 02 — Playwright Core — UI & API Testing

Master end-to-end UI flows AND API validation in one framework

Playwright Setup	– Installation, project scaffold & folder structure – Browser contexts, pages, fixtures & hooks – playwright.config.ts deep dive — envs, retries, timeouts
UI Testing	– Locators & selectors — best practices & anti-patterns – Actions: click, fill, drag, upload, keyboard & mouse events – Assertions — expect(), soft assertions & auto-waiting – Screenshots, videos & trace on failure
API Testing	– APIRequestContext — GET, POST, PUT, DELETE – Request/response schema validation – Auth token injection & header management – Combining UI + API in a single test flow
Cross-Browser & Mobile	– Running tests on Chromium, Firefox & WebKit – Mobile device emulation — viewport, touch, geolocation – Playwright device descriptor library – Responsive layout testing strategies
Test Organization	– Tags, grep filters & test.describe blocks – Before/After hooks — setup & teardown patterns – Parallel execution, sharding & retries

Module 03 — BDD with Cucumber & Gherkin

Bridge the gap between business requirements and test code

BDD Foundations	– Why BDD? Business value of readable test scenarios – Gherkin syntax — Feature, Scenario, Given/When/Then/And – Scenario Outline & Examples for data-driven BDD
Cucumber Setup	– cucumber-js setup with Playwright + TypeScript – Writing Step Definitions — mapping Gherkin to actions – World object & shared state across steps – Hooks — Before, After, BeforeAll, AfterAll
Advanced BDD	– Tags & filtering — @smoke, @regression, @wip – Background steps & shared preconditions – DocString & DataTable usage in steps – HTML + JSON report output from Cucumber

Module 04 — Page Object Model (POM) Architecture

Build maintainable, reusable & scalable test structure

POM Design	– BasePage class — shared methods & constructor patterns – Page-specific classes & locator strategy – Lazy locator pattern & element encapsulation – Avoiding anti-patterns — no logic in tests, no hardcoded selectors
API Client Layer	– Reusable API request builder class – Response schema validation – Auth token management & session handling – Chaining API + UI in POM flows
Clean Architecture	– Enforcing folder structure across the team – Dependency injection via Playwright fixtures – Environment config management — .env + config files – Code review checklist for test architecture

Module 05 — AI-Powered Test Intelligence

Smart test data, self-healing locators & AI-driven defect insights

Smart Test Data	– AI-generated test data using LLMs (GPT-4o, Claude, Gemini) – Dynamic edge case generation for boundary testing – Seeding DB with AI-generated realistic datasets
Self-Healing Tests	– Locator failure detection & LLM-powered auto-fix flow – GitHub PR auto-creation for healed test files – Live demo — real-world app flaky test fix
Defect Insights	– AI-powered failure log analysis & root cause summarization – Integrating AI analysis into CI pipeline output – Auto-generate Gherkin scenarios from requirements doc – Summarise test run results via LLM

Module 06 — MCP Server + Playwright Agent

Connect AI to your tools, data & test actions — the future of automation

MCP Fundamentals	– What is MCP? Protocol & architecture explained – How AI connects to external tools via MCP layer – MCP vs traditional API integration — when to use what
Playwright MCP Server	– @playwright/mcp — official Playwright MCP server setup – Giving AI agents full browser control via Playwright – Claude, GPT-4o, Gemini — any LLM using Playwright as a tool – Natural language to browser action pipeline
Building MCP Layers	– Connecting AI assistant to your Playwright test suite – MCP + DB, Docs & API as intelligent data sources – Prompt engineering for test execution actions
Live Agent Demo	– End-to-end: User prompt to AI to MCP to Browser to Result – AI agent generating & running tests automatically – Analysing failures and self-correcting via MCP loop

Module 07 — CI/CD Pipeline Integration

Quality gates in every build — no broken code ships

Pipeline Concepts	– CI/CD fundamentals for QA engineers – Flow: Code Commit > Build > Run Tests > Report > Quality Gate – Branching strategies & test trigger rules
Jenkins + GitHub	– Jenkins setup & Playwright pipeline config – GitHub Actions .yaml workflows for Playwright + Cucumber – Matrix testing — multi-browser in parallel – PR quality gate — block merge on test failure
Allure Reporting	– Allure setup with Playwright + Cucumber output – Rich HTML reports — trends, history & attachments – Publishing Allure reports in Jenkins & GitHub Actions – Scheduled nightly regression runs

- ✓ Integrate AI to self-heal broken tests, generate smart test data & analyse failures automatically
- ✓ Use @playwright/mcp to give AI agents full browser control — the future of test automation
- ✓ Set up end-to-end CI/CD pipelines — Jenkins & GitHub Actions with quality gate enforcement
- ✓ Publish rich Allure HTML reports with trends, history & screenshot attachments in CI

TAKEAWAY

Combining Playwright + TypeScript + BDD with AI & MCP creates an intelligent, scalable and future-ready test automation ecosystem.

Let's build quality that scales with innovation!

#Playwright #TypeScript #BDD #AI #MCP #TestAutomation #QualityEngineering